

CRASH COURSE PLAN — v2 (đã audit)

NestJS Crash Course

Lộ trình học chi tiết từng phần — kèm giải thích cơ chế hoạt động bên trong

Repo: [crash-course / nestjs-crash-course](#)

Bản audit: đối chiếu lại với code thật đã chạy qua Part 1-9, sửa các điểm sai/thiếu, bổ sung Part 12-13

Mục lục

00 Kiến trúc tổng thể & Request Lifecycle

01 Khởi tạo & cấu trúc dự án

02 Controllers & Routing

03 Providers, Services & Dependency Injection

04 Modules

05 DTO & Validation

06 Pipes & Exception Filters

07 Middleware, Guards, Interceptors

08 Kết nối Database với TypeORM

09 Quan hệ dữ liệu (Relations)

10 Authentication (JWT + Passport)

11 Testing (Jest)

12 Pagination — MỚI

13 Transactions — MỚI

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh họa để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Bản audit này khác gì bản gốc: mọi phần đã được đối chiếu lại với code THẬT đã chạy qua trong quá trình học (Part 1-9), sửa các chỗ sai do phiên bản package khác (đặc biệt Part 08), bổ sung các bài học rút ra từ lỗi thực tế đã gặp (đánh dấu bằng khung màu xanh này), và thêm 2 phần mới: Pagination, Transactions.

Kiến trúc tổng thể & Request Lifecycle

Đây là phần quan trọng nhất để hiểu "mọi thứ hoạt động thế nào" trước khi đi vào từng phần nhỏ. NestJS không phải một framework hoàn toàn mới — nó là một lớp kiến trúc (architecture layer) xây trên nền HTTP server có sẵn (mặc định là **Express**, có thể đổi sang **Fastify**). NestJS lấy cảm hứng từ Angular: dùng **TypeScript decorators**, **Dependency Injection (DI)**, và tổ chức code theo **Module**.

1. Decorator & Reflection Metadata — "phép màu" đằng sau NestJS

Toàn bộ NestJS hoạt động dựa trên một cơ chế của TypeScript gọi là **decorator** kết hợp thư viện **reflect-metadata**. Khi bạn viết:

```
@Controller('posts')
export class PostsController { }
```

Decorator `@Controller('posts')` thực chất là một hàm chạy tại thời điểm class được định nghĩa (compile/load time). Nó không "chạy logic nghiệp vụ" — nó chỉ **gắn metadata** (dữ liệu mô tả) vào class đó, kiểu như dán nhãn: "class này là một Controller, prefix route là `/posts`". NestJS lưu metadata này bằng `Reflect.defineMetadata()`.

Khi ứng dụng khởi động (`bootstrap()`), NestJS quét toàn bộ metadata đã được gắn (dùng `Reflect.getMetadata()`) để biết: class nào là Controller, class nào là Provider, route nào ứng với method nào, cần inject dependency gì vào constructor... Đây chính là lý do NestJS gọi là "declarative" — bạn *khai báo* ý định bằng decorator, framework tự đọc và ráp nối lại.

Bài học thực tế: vì decorator/class chỉ là 1 identifier bình thường trong TypeScript, việc đặt tên Entity/DTO trùng tên với 1 decorator import từ `@nestjs/common` (ví dụ đặt Entity tên `Post`, trùng decorator `@Post()`) là cái bẫy rất dễ gặp — xem cảnh báo chi tiết ở Part 08.

2. Module Graph & Dependency Injection Container (IoC Container)

Khi app khởi động, NestJS xây dựng một **cây module** (module tree) bắt đầu từ `AppModule` (root module). Với mỗi module, nó đọc metadata trong `@Module({...})`:

- `imports`: các module khác mà module này cần dùng
- `controllers`: các controller thuộc module này
- `providers`: các service/class sẽ được đăng ký vào DI container
- `exports`: các provider mà module này "cho phép" module khác dùng lại

NestJS tạo ra một **IoC Container** (Inversion of Control) — một "kho chứa instance". Mỗi provider (mặc định là **singleton**) chỉ được khởi tạo **một lần duy nhất** trong toàn bộ vòng đời app. Khi class A cần class B trong constructor:

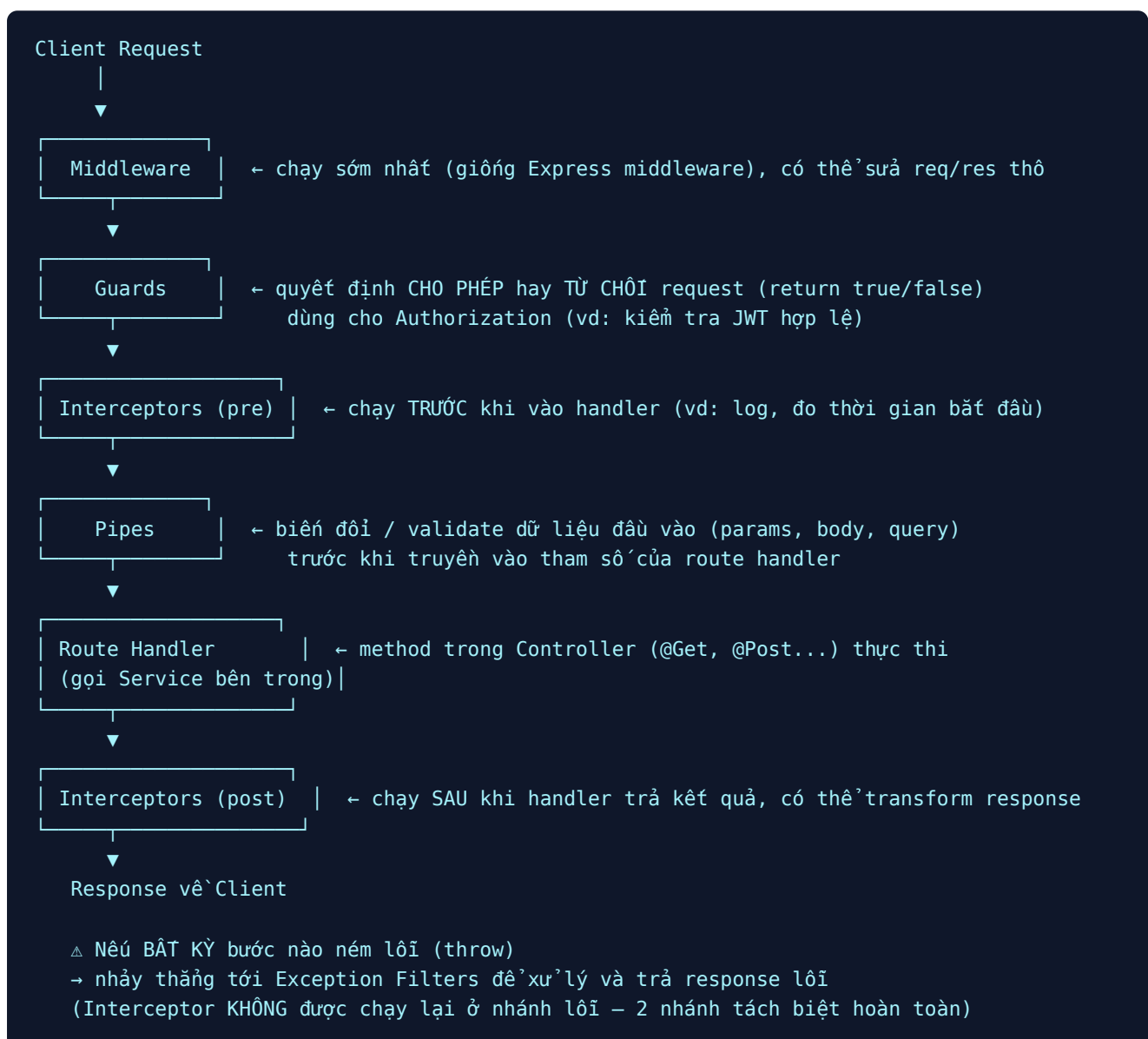
```
@Injectable()
export class PostsController {
  constructor(private readonly postsService: PostsService) {}
}
```

NestJS nhìn vào **type** của tham số constructor (nhờ TypeScript compile ra metadata kiểu dữ liệu), tra trong container xem đã có instance của `PostsService` chưa — nếu có thì tái sử dụng, nếu chưa thì tạo mới rồi lưu lại. Đây gọi là **constructor-based injection**. Bạn không bao giờ tự viết `new PostsService()` — NestJS làm việc đó cho bạn. Lợi ích: dễ test (mock được), dễ thay thế implementation, giảm coupling giữa các class.

Encapsulation theo module: Provider trong module A *không* tự động dùng được ở module B — trừ khi A `export` nó ra và B `import` module A vào. Đây là ranh giới đóng gói (giống package-private), giúp kiến trúc lớn không bị rối.

3. Request Lifecycle — thứ tự một request đi qua hệ thống

Đây là sơ đồ quan trọng nhất cần thuộc lòng. Khi một HTTP request đến, nó đi qua các lớp theo đúng thứ tự sau:



Sơ đồ này trả lời hầu hết câu hỏi "tại sao": Guard chạy trước Controller để chặn truy cập trái phép sớm; Pipe chạy trước handler để data luôn sạch/hợp lệ; Interceptor "bọc" được cả input lẫn output vì nó chạy cả trước và sau.

4. Provider Scope (phạm vi sống của instance)

99% trường hợp trong crash course này bạn sẽ dùng Singleton (mặc định) — không cần khai báo gì thêm. Bảng dưới để biết còn 2 lựa chọn khác khi cần:

Scope	Mặc định?	Giải thích
DEFAULT (Singleton)	✓ Có	Một instance duy nhất, dùng chung cho toàn bộ app suốt vòng đời server
REQUEST	Không	Tạo instance mới cho mỗi request — dùng khi cần lưu state riêng theo request (vd: multi-tenant)
TRANSIENT	Không	Tạo instance mới mỗi lần được inject — hiếm dùng

PART 01

Khởi tạo & cấu trúc dự án

Mục tiêu: Hiểu Nest CLI tạo ra những gì, và luồng khởi động (bootstrap) của một app NestJS.

1.1 Cài đặt & tạo project

```
npm i -g @nestjs/cli
nest new nestjs-crash-course
```

Nest CLI dùng schematics (khuôn mẫu code) để sinh ra cấu trúc chuẩn, tương tự `create-react-app` hay `vue create`.

1.2 Cấu trúc thư mục mặc định

File	Vai trò
src/main.ts	Entry point — nơi app thực sự khởi động
src/app.module.ts	Root Module — gốc của cây module
src/app.controller.ts	Controller mẫu, xử lý route <code>/</code>
src/app.service.ts	Service mẫu, chứa logic nghiệp vụ
nest-cli.json	Cấu hình Nest CLI (build, schematics...)
tsconfig.json	Cấu hình TypeScript — chú ý <code>experimentalDecorators</code> và <code>emitDecoratorMetadata</code> phải bật (đây là 2 cờ TS bắt buộc để decorator + DI hoạt động)

Lưu ý về `tsconfig.json` : nếu không khai báo tường minh mảng `"lib"` , TypeScript sẽ tự nạp thêm cả bộ type của **DOM** (dành cho browser) bên cạnh target ES bạn chọn. Điều này gây ra 1 bug rất khó phát hiện ở Part 09 (biến toàn cục `Comment` của DOM trùng tên với Entity tự viết) — cứ để mặc định lúc học, nhưng nhớ lại phần này khi gặp lỗi "tsc sạch nhưng chạy vẫn lỗi".

1.3 Giải phẫu main.ts

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` chính là bước NestJS đọc `AppModule` , duyệt toàn bộ `imports` để quy, đăng ký hết Controllers/Providers vào IoC container, resolve toàn bộ dependency injection — tất cả xảy ra **trước** khi `listen()` được gọi. Nếu có lỗi thiếu dependency hoặc circular dependency, app sẽ crash ngay tại bước `create()` này (rất hữu ích để debug).

Việc cần làm:

1. Tạo project bằng `nest new nestjs-crash-course`
2. Chạy `npm run start:dev` , mở `http://localhost:3000` , xác nhận thấy "Hello World!"
3. Đọc kỹ 4 file: `main.ts`, `app.module.ts`, `app.controller.ts`, `app.service.ts` — thử tự vẽ lại sơ đồ ai gọi ai
4. Thử đổi cổng bằng biến môi trường `PORT=4000 npm run start:dev`

PART 02

Controllers & Routing

Mục tiêu: Hiểu cách NestJS map một HTTP request tới đúng method trong Controller, và cách lấy dữ liệu từ request.

2.1 Cơ chế map route

`@Controller('posts')` đăng ký prefix `/posts` . Mỗi method bên trong gắn thêm decorator HTTP method:

```
@Controller('posts')
export class PostsController {
  @Get() // GET /posts
  @Get(':id') // GET /posts/:id
  @Post() // POST /posts
  @Put(':id') // PUT /posts/:id
  @Delete(':id') // DELETE /posts/:id
}
```

Khi bootstrap, NestJS quét toàn bộ Controller đã đăng ký trong `controllers: []` của module, đọc metadata của từng method, rồi đăng ký route đó thẳng vào Express router bên dưới. Về bản chất, NestJS chỉ là một lớp "khai báo" đặt lên trên `app.get()`, `app.post()` của Express.

2.2 Param Decorators — lấy dữ liệu từ request

Decorator	Lấy gì	Ví dụ
<code>@Param('id')</code>	Route param	<code>/posts/5</code> → <code>id = "5"</code>
<code>@Query('search')</code>	Query string	<code>/posts?search=abc</code> → <code>"abc"</code>
<code>@Body()</code>	Toàn bộ request body (JSON)	dùng cho POST/PUT
<code>@Headers('authorization')</code>	HTTP header	lấy token
<code>@Req()</code>	Object request thô của Express	ít dùng, phá vỡ tính "framework-agnostic"

Các decorator này thực chất cũng chỉ là metadata — NestJS đọc metadata đó, biết phải lấy giá trị nào từ `req` gốc của Express, rồi **tự động truyền đúng thứ tự** vào tham số hàm của bạn khi gọi handler. Đây là lý do bạn không cần viết `req.params.id` thủ công như Express thuần.

Đúc kết từ thực tế: `@Param('id')` LUÔN trả về `string`, kể cả khi route param là số. Route `GET /posts/:id` viết đơn giản nhất chỉ nhận `id: string` rồi tự `Number(id)` — nhưng cách chuẩn hơn (Part 06) là gắn `ParseIntPipe` ngay tại decorator để Nest tự ép kiểu, tự trả 400 nếu id không phải số. Nếu bạn thấy so sánh `post.id === id` luôn `false` dù nhìn "giống nhau" trên console.log — 99% là do 1 bên `number` 1 bên `string`.

Việc cần làm:

1. Tạo `PostsController` bằng `nest g controller posts`
2. Viết 5 route CRUD cơ bản (GET all, GET one, POST, PUT, DELETE) — trả về mảng cứng trong bộ nhớ (chưa cần service/database)
3. Test bằng Postman/curl/Thunder Client cho từng route

PART 03

Providers, Services & Dependency Injection

Mục tiêu: Tách logic nghiệp vụ ra khỏi Controller, hiểu sâu cách DI container resolve dependency.

3.1 Tại sao tách Service?

Nguyên tắc **Separation of Concerns**: Controller chỉ nên lo việc "nhận request → gọi service → trả response". Toàn bộ logic nghiệp vụ (tính toán, thao tác dữ liệu, gọi database) nằm trong Service. Điều này giúp: Controller dễ đọc, Service tái sử dụng được ở nơi khác (vd: một cron job cũng gọi lại service đó), và quan trọng nhất — **dễ test** (test Service không cần giả lập HTTP request).

3.2 @Injectable() nghĩa là gì?

```
@Injectable()
export class PostsService {
  private posts = [ ... ];
  findAll() { return this.posts; }
}
```

`@Injectable()` đánh dấu class này **có thể được đưa vào DI container** để inject cho class khác. Bản thân từ "Injectable" nghĩa là "có thể tiêm vào" — không phải class này tự tiêm, mà là nó *đủ điều kiện để bị tiêm vào constructor của class khác*.

3.3 Luồng resolve dependency chi tiết

1. App bootstrap → NestJS đọc `@Module({ providers: [PostsService] })`
2. NestJS thấy `PostsController` có constructor(`private postsService: PostsService`)
3. NestJS tra IoC container: "đã có instance `PostsService` chưa?"
 - Chưa có → tạo mới `new PostsService()` → LƯU VÀO container (singleton)
 - Đã có → lấy lại instance cũ, KHÔNG tạo mới
4. NestJS gán instance đó vào `this.postsService` của `PostsController`
5. Toàn bộ app dùng CHUNG 1 instance `PostsService` này

Vì sao phải khai báo providers trong @Module? Nếu bạn quên thêm `PostsService` vào mảng `providers` của module, NestJS sẽ báo lỗi `Nest can't resolve dependencies` ngay lúc khởi động — vì nó không biết class đó tồn tại trong container.

Việc cần làm:

1. Tạo `PostsService` bằng `nest g service posts`
2. Chuyển toàn bộ logic (mảng posts, các hàm CRUD) từ Controller sang Service
3. Inject Service vào Controller qua constructor, Controller chỉ gọi lại các method của Service

PART 04

Modules

Mục tiêu: Tổ chức app lớn theo feature module thay vì nhồi hết vào AppModule.

4.1 Feature Module là gì?

Khi app lớn dần, bạn không nhét tất cả Controller/Service vào `AppModule`. Thay vào đó, mỗi "tính năng" (feature) có module riêng — vd `PostsModule`, `UsersModule`, `AuthModule` — rồi `AppModule` chỉ `import` các module con này lại.

```
@Module({
  controllers: [PostsController],
  providers: [PostsService],
  exports: [PostsService], // cho module khác dùng lại PostsService
})
export class PostsModule {}
```

`nest g resource posts` là lệnh "tất cả trong một" — tự sinh Controller, Service, Module, DTO, Entity cùng lúc và tự động import vào AppModule. Rất hay dùng trong thực tế.

Việc cần làm:

1. Refactor `PostsController` / `PostsService` đã làm ở Part 2-3 vào một `PostsModule` riêng
2. Import `PostsModule` vào `AppModule`, xóa bớt khai báo trùng lặp ở AppModule
3. Thử tạo thêm 1 module mới bằng `nest g resource users` để quen tay với CLI

PART 05

DTO & Validation

Mục tiêu: Đảm bảo dữ liệu client gửi lên luôn đúng định dạng trước khi vào tới Service.

5.1 DTO (Data Transfer Object) là gì?

DTO là một class mô tả "hình dạng" dữ liệu được truyền giữa client và server (khác với Entity — mô tả bằng trong database, sẽ học ở Part 8). Ví dụ `CreatePostDto` mô tả body hợp lệ khi tạo post mới.

5.2 class-validator + ValidationPipe hoạt động thế nào

```
export class CreatePostDto {
  @IsString() @NotEmpty()
  title: string;
  @IsString() @NotEmpty()
  body: string;
}
```

Decorator như `@IsString()` gắn metadata validation lên từng field (thư viện `class-validator`). Khi request tới, `ValidationPipe` (chạy ở bước "Pipes" trong request lifecycle — xem Part 0) sẽ:

1. Lấy raw JSON body, `class-transformer` biến nó thành instance thật của class DTO (`plainToInstance`)
2. `class-validator` chạy tất cả rule decorator đã khai báo, thu thập lỗi
3. Nếu có lỗi → tự động throw `BadRequestException` (HTTP 400) kèm danh sách lỗi chi tiết, request **không bao giờ chạm tới Controller**
4. Nếu hợp lệ → object sạch được truyền vào `@Body()` của handler

Đăng ký global (áp dụng cho toàn bộ app) trong `main.ts`:

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true, // tự động loại bỏ field lạ không khai báo trong DTO
  forbidNonWhitelisted: true, // field lạ → throw lỗi thay vì âm thầm bỏ qua
}));
```

Việc cần làm:

1. Cài `class-validator` `class-transformer`
2. Tạo `CreatePostDto` và `UpdatePostDto` (dùng `PartialType` từ `@nestjs/mapped-types` để field đều optional)
3. Gắn `ValidationPipe` global trong `main.ts`
4. Test gửi body thiếu field / sai kiểu dữ liệu → xác nhận nhận về lỗi 400 rõ ràng

PART 06

Pipes & Exception Filters

Mục tiêu: Tự viết pipe/filter riêng, hiểu cách NestJS xử lý lỗi tập trung.

6.1 Custom Pipe / Pipe có sẵn

Pipe là class implement interface `PipeTransform` với method `transform(value, metadata)` — nhận giá trị đầu vào, trả về giá trị đã biến đổi (hoặc throw lỗi nếu không hợp lệ). Pipe có sẵn hay dùng nhất:

`ParseIntPipe` — ép `:id` từ string sang number ngay tại tầng Pipe, tự throw `BadRequestException` nếu không parse được:

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number) {
  // id đã là number thật, không cần Number(id) nữa
  return this.postsService.findOne(id);
}
```

6.2 Exception hierarchy

NestJS có sẵn class `HttpException` và các class con tương ứng mã HTTP: `NotFoundException` (404), `BadRequestException` (400), `UnauthorizedException` (401), `ForbiddenException` (403)... Khi bạn `throw new NotFoundException('Post not found')` ở bất kỳ đâu (Service hay Controller), NestJS có sẵn một **Exception Filter mặc định** bắt lấy, format thành JSON response chuẩn và trả về client — bạn không cần try/catch thủ công trong từng route.

6.3 Custom Exception Filter — bản đã audit

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();
    const status = exception.getStatus();

    // getResponse() trả về `string` (khi throw new NotFoundException('msg'))
    // HOẶC object { message, error, statusCode } (khi Nest tự tạo payload).
    // Phải bóc ra, nếu không client sẽ phải đọc body.message.message.
    const payload = exception.getResponse();
    const message =
      typeof payload === 'string'
        ? payload
        : (payload as { message?: string }).message ?? exception.message;

    response.status(status).json({
      success: false, // khớp key "success" với TransformInterceptor ở Part 07
      statusCode: status,
      path: request.url,
      timestamp: new Date().toISOString(),
      message,
    });
  }
}
```

Bài học từ thực tế: phiên bản đầu tiên (chỉ nhét nguyên `exception.getResponse()` vào field `message`) tạo ra response lồng 2 lớp `message: { message: '...', error, statusCode }` — rất khó dùng ở client. `getResponse()` có thể trả về `string` (khi bạn tự `throw new NotFoundException('text')`) hoặc `object` (khi Nest tự sinh payload lỗi mặc định) — luôn phải kiểm tra `typeof` trước khi dùng. Field `success: false` ở đây cũng nên khớp với `success: true` mà Interceptor gán ở Part 07, để client chỉ cần check 1 field duy nhất cho cả 2 nhánh thành công/lỗi.

`@Catch(HttpException)` nghĩa là filter này chỉ bắt lỗi thuộc loại `HttpException` (và các class con) — nếu bạn `throw new Error(...)` thường (không kế thừa `HttpException`), filter này **không bắt được**, sẽ rơi vào handler lỗi 500 mặc định của Nest. Đây là lý do luôn dùng `NotFoundException / BadRequestException` ... thay vì tự `throw new Error()`.

Việc cần làm:

1. Trong `PostsService`, khi không tìm thấy post theo id → throw `NotFoundException`
2. Gắn `ParseIntPipe` vào mọi route nhận `:id`
3. Viết exception filter như trên, đăng ký global bằng `app.useGlobalFilters()`

Middleware, Guards, Interceptors

Mục tiêu: Nắm 3 lớp còn lại trong request lifecycle, biết dùng đúng công cụ cho đúng việc.

7.1 Middleware — chạy sớm nhất, gần Express nhất

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    const start = Date.now();
    res.on('finish', () => {
      const ms = Date.now() - start;
      console.log(`${req.method} ${req.originalUrl} ${res.statusCode} - ${ms}ms`);
    });
    next(); // bắt buộc gọi next() để đi tiếp, nếu không request sẽ treo
  }
}
```

Tại sao log trong `res.on('finish')` chứ không log ngay sau `next()` : ngay sau `next()` , response chưa thực sự được gửi xong — `res.statusCode` lúc đó vẫn chưa phải giá trị cuối cùng. Lắng nghe event `'finish'` đảm bảo log đúng status code thật và đo được thời gian xử lý toàn bộ request.

Middleware KHÔNG khai báo trong `@Module({...})` như Controller/Provider — nó được gắn thủ công qua method `configure()` trong module, dùng `MiddlewareConsumer` để chỉ định áp dụng cho route nào (`consumer.apply(LoggerMiddleware).forRoutes('*')` = áp dụng toàn app).

7.2 Guards — trả lời câu hỏi "có được vào không?"

```
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    if (!request.headers.authorization) {
      throw new UnauthorizedException('Missing Authorization header');
    }
    return true;
  }
}
```

Guard implement `CanActivate` , trả về `true` / `false` (hoặc `Promise<boolean>`) — mặc định trả `false` sẽ bị Nest tự trả 403; ở đây chủ động `throw UnauthorizedException` để kiểm soát status 401 rõ ràng hơn. Đây là nơi chuẩn để đặt logic **Authorization** — khác với Middleware ở chỗ Guard có quyền truy cập `ExecutionContext` (biết được đang gọi tới Controller/handler nào, hữu ích để đọc thêm metadata custom qua `Reflector` , ví dụ decorator `@Roles('admin')` tự viết).

7.3 Interceptors — bọc quanh cả input lẫn output

```
@Injectable()
export class TransformInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    return next.handle().pipe(
      map(data => ({ success: true, data })))
  };
}
```

Interceptor dùng **RxJS Observable**. `next.handle()` trả về Observable đại diện cho kết quả của route handler — bạn có thể `.pipe(map(...))` để biến đổi response trước khi trả về client. Đăng ký global bằng `app.useGlobalInterceptors(new TransformInterceptor())` trong `main.ts`.

Đã kiểm chứng bằng test thật: sau khi bật Interceptor này, response LỖI (từ `HttpExceptionHandler` Part 06) **không** bị bọc theo format `{success, data}` — vì Interceptor chỉ bọc nhánh `next.handle()` (thành công), nhánh lỗi đi thẳng qua Exception Filter, bỏ qua Interceptor hoàn toàn (đúng như sơ đồ Part 00). Đây là lý do `HttpExceptionHandler` phải tự thêm field `success: false` bằng tay — không có Interceptor nào làm việc đó cho nhánh lỗi.

7.4 So sánh nhanh

Công cụ	Dùng khi nào	Chạy ở đâu trong lifecycle
Middleware	Logging, đọc/sửa raw request sớm	Sớm nhất, trước routing
Guard	Authorization (được phép truy cập không?)	Sau routing, trước Interceptor
Interceptor	Transform response, logging có ngữ cảnh handler, cache	Bọc quanh route handler
Pipe	Validate/transform input	Ngay trước khi vào handler
Exception Filter	Xử lý lỗi tập trung	Bất cứ khi nào có exception

Việc cần làm:

- Viết `LoggerMiddleware` log method + url + status + thời gian, áp dụng cho toàn bộ route
- Viết `AuthGuard` đơn giản kiểm tra header `Authorization` có tồn tại, áp lên route tạo/sửa/xoá post
- Viết `TransformInterceptor` bọc mọi response thành công theo format `{ success: true, data }`, đăng ký global

Kết nối Database với TypeORM

Mục tiêu: Thay mảng dữ liệu cứng trong bộ nhớ bằng database thật (SQLite để đơn giản, có thể nâng cấp Postgres sau).

8.1 TypeORM là gì

TypeORM là một **ORM** (Object-Relational Mapping) — cho phép thao tác database bằng object/class TypeScript thay vì viết SQL tay. NestJS tích hợp sẵn qua package `@nestjs/typeorm`.

⚠ **Đã sửa so với bản gốc:** bản trước hướng dẫn dùng `type: 'sqlite'` + package `sqlite3` — đây là API TypeORM Cũ. Bản TypeORM hiện tại (kiểm tra bằng `npm view typeorm version`) đã **bỏ hẳn driver "sqlite"**, chỉ còn `"better-sqlite3"` cho SQLite. Luôn kiểm tra `node_modules/typeorm/driver/types/DatabaseType.d.ts` nếu nghi ngờ danh sách driver hợp lệ đã đổi.

8.2 Entity — class ánh xạ tới bảng

```
@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @Column()
  body: string;
}
```

`@Entity()` đánh dấu class này ánh xạ tới một bảng — TypeORM tự đặt tên bảng bằng cách lowercase tên class (`Post` → bảng `post`, KHÔNG tự thêm "s"). Mỗi `@Column()` là một cột. TypeORM đọc metadata này để tự sinh câu SQL `CREATE TABLE` (nếu bật `synchronize: true` — chỉ nên dùng khi dev, không dùng ở production vì có thể mất dữ liệu khi đổi schema).

⚠ **Bẫy trùng tên — RẤT dễ gặp:** Entity thường được đặt tên số ít khớp domain (`Post`, `Comment`, `User`) — nhưng `Post` lại trùng tên với decorator `@Post()` import từ `@nestjs/common`! Trong Controller/Service cần dùng cả 2, phải **alias khi import**:

```
import { Post as PostEntity } from './entities/post.entity';
```

Nếu bạn thấy TypeScript báo `error TS2749: 'Post' refers to a value, but is being used as a type here` — chính xác là bug này. Alias theo chiều nào (đổi tên decorator hay Entity) tùy bạn, chỉ cần nhất quán trong 1 file.

8.3 Repository Pattern

```
constructor(  
  @InjectRepository(PostEntity)  
  private readonly postsRepository: Repository<PostEntity>,  
) {}  
  
// find() Đã tự trả Promise → chỉ forward ra, KHÔNG cần async  
findAll(): Promise<PostEntity[]> {  
  return this.postsRepository.find();  
}  
  
// BẮT BUỘC async/await: cần giá trị thật của post để kiểm tra if (!post)  
async findOne(id: number): Promise<PostEntity> {  
  const post = await this.postsRepository.findOneBy({ id });  
  if (!post) {  
    throw new NotFoundException(`Post with id ${id} not found`);  
  }  
  return post;  
}  
  
create(dto: CreatePostDto): Promise<PostEntity> {  
  // create() ĐÔNG BỘ: chỉ tạo object JS trong RAM, chưa chạm DB  
  const post = this.postsRepository.create(dto);  
  // save() BẮT ĐÔNG BỘ: trả Promise → return thẳng, không cần await  
  return this.postsRepository.save(post);  
}
```

`@InjectRepository(PostEntity)` là một biến thể đặc biệt của DI — thay vì inject trực tiếp một class do bạn viết, bạn inject một **Repository object** mà TypeORM tự sinh ra cho Entity `PostEntity`. Repository này có sẵn các method `find`, `findOneBy`, `save`, `remove` ... — về bản chất đây vẫn là DI container, chỉ là "provider" ở đây được đăng ký bởi `TypeOrmModule.forFeature([Post])` thay vì bạn tự viết `@Injectable()`.

Quy tắc chung khi nào cần `async/await` : chỉ cần khi bạn phải dùng giá trị đã resolve của 1 Promise ngay trong thân hàm (so sánh `if`, `Object.assign`, truyền làm tham số cho hàm khác cần entity thật...). Nếu chỉ nhận 1 Promise rồi trả thẳng ra ngoài (không dùng vào giá trị) — không cần `async/await`, cứ `return` Promise đó, người gọi tự lo. Ví dụ cụ thể: `findAll()` / `create()` không cần `async` (chỉ forward Promise), `findOne()` / `update()` / `remove()` bắt buộc cần (phải `if (!post)`, hoặc cần entity thật để gọi `repository.remove(entity)`).

Bug nguy hiểm nếu quên await ở Controller:

```
// SAI – không await, không return:
remove(id: number) {
  this.postsService.remove(id); // Promise bị "bỏ rơi"
  return { message: 'deleted' }; // trả response NGAY, chưa chắc đã xoá xong
}
```

Nếu `id` không tồn tại, `service.remove()` sẽ **throw** bên trong 1 Promise không ai **await** — thành "unhandled promise rejection", Nest không bắt được, và tùy phiên bản Node.js có thể **crash cả server**. Luôn **await** Promise trước khi return trong Controller.

Việc cần làm:

1. Cài `@nestjs/typeorm typeorm better-sqlite3` (KHÔNG dùng `sqlite3` — xem cảnh báo ở trên)
2. Cấu hình `TypeOrmModule.forRoot({ type: 'better-sqlite3', database: 'db.sqlite3', autoLoadEntities: true, synchronize: true })` trong AppModule
3. Tạo `Post` entity thay cho mảng cứng, nhớ alias khi import vào Controller/Service (mục 8.2)
4. Đăng ký `TypeOrmModule.forFeature([Post])` trong PostsModule, inject Repository vào PostsService
5. Chuyển toàn bộ hàm CRUD trong service sang gọi Repository, thêm `async/await` đúng chỗ (mục 8.3)
6. Thêm `db.sqlite3` vào `.gitignore` — không commit file database lên git
7. Kiểm chứng persistence: tạo 1 post, restart hẳn server (`Ctrl+C` rồi chạy lại), `GET` lại — post phải còn nguyên

PART 09

Quan hệ dữ liệu (Relations)

Mục tiêu: Thêm entity thứ 2 (Comment) và liên kết với Post — hiểu cách TypeORM quản lý khoá ngoại.

9.1 One-to-Many / Many-to-One

```
// Trong Post entity
@OneToMany(() => Comment, (comment) => comment.post)
comments: Comment[];

// Trong Comment entity
@ManyToOne(() => Post, (post) => post.comments, { onDelete: 'CASCADE' })
post: Post;
```

Một `Post` có nhiều `Comment` (OneToMany), mỗi `Comment` thuộc về đúng một `Post` (ManyToOne) — hai decorator này luôn đi thành cặp, tham số thứ 2 là hàm trả ngược lại field ở phía bên kia để TypeORM biết đây

là 2 đầu của cùng 1 quan hệ. **Khoá ngoại thật (postId) chỉ nằm ở bảng phía "Many" (comment)** — @OneToMany không tạo cột nào cả. onDelete: 'CASCADE' — xoá Post thì mọi Comment của nó tự bị xoá theo, tránh dữ liệu "mồ côi".

⚠ **Bẫy cực tinh vi — global Comment của DOM:** nếu bạn import { Comment } from './entities/comment.entity' nhưng lỡ quên import ở nơi khác dùng đến (ví dụ trong @Module({ imports: [TypeOrmModule.forFeature([Comment, Post])] })), TypeScript **sẽ không báo lỗi gì!** Lý do: khi tsconfig.json không giới hạn "lib", TypeScript tự nạp thêm type của DOM — trong đó có sẵn 1 biến toàn cục tên Comment (constructor của "comment node" trong HTML, giống Document). Compile "sạch", nhưng chạy trong Node.js sẽ ReferenceError: Comment is not defined ngay khi module được nạp — vì Node.js không có global đó. **Bài học:** "tsc không báo lỗi" không đồng nghĩa "chắc chắn đúng" khi có tên trùng global ambient type/value có sẵn trong lib mặc định. Luôn để ý gợi ý "Add import" của IDE khi gõ tên class — nếu nó không hiện gợi ý dù bạn chưa import, hãy nghi ngờ.

9.2 Eager vs Lazy loading

eager: true trong decorator quan hệ nghĩa là mỗi lần query Post, TypeORM tự động JOIN kèm luôn Comment. Nếu không bật, bạn phải chủ động yêu cầu bằng relations: { comments: true } khi query — cách này kiểm soát hiệu năng tốt hơn (tránh JOIN thừa khi không cần, ví dụ GET /posts danh sách không cần tải hết comments của từng post).

Đã sửa so với bản gốc: bản trước viết relations: ['comments'] (mảng chuỗi) — đây là API TypeORM CŨ. Bản TypeORM hiện tại chỉ chấp nhận object: relations: { comments: true }, lồng sâu thì { comments: { author: true } }. Dùng mảng chuỗi sẽ báo lỗi compile Type 'string[]' has no properties in common with type 'FindOptionsRelations<Post>'.

9.3 Kiểm tra parent tồn tại trước khi tạo con

```
async create(postId: number, dto: CreateCommentDto): Promise<Comment> {
  const post = await this.postsRepository.findOneBy({ id: postId });
  if (!post) {
    throw new NotFoundException(`Post with id ${postId} not found`);
  }
  // gán cả object post vào field quan hệ, KHÔNG phải postId
  const comment = this.commentsRepository.create({ ...dto, post });
  return this.commentsRepository.save(comment);
}
```

Gán trực tiếp object post (không phải postId) vào field quan hệ — TypeORM tự hiểu cần lưu post.id vào cột khoá ngoại bên dưới. Luôn kiểm tra parent tồn tại trước — nếu không sẽ tạo ra comment "mồ côi" ngay từ đầu ở tầng nghiệp vụ (dù DB không cấm).

9.4 Đếm số bản ghi liên quan — không tải cả mảng chỉ để lấy .length

Bài toán rất hay gặp: category có bao nhiêu product, post có bao nhiêu comment... **Cách sai** — tải nguyên bảng con rồi đếm:

```
// SAI – kéo hết dữ liệu product (name, price, description...) chỉ để đếm
const category = await this.categoriesRepository.findOne({
  where: { id },
  relations: { products: true },
});
const count = category.products.length; // category có 10.000 product = kéo cả 10.000 dòng
```

Cách đúng: yêu cầu database tự đếm (**COUNT**), không kéo dữ liệu con về. TypeORM có 2 hàm cho việc này trên **QueryBuilder** — nhưng **chỉ 1 hàm còn tồn tại** ở bản bạn đang cài:

```
const [rows, total] = await this.categoriesRepository
  .createQueryBuilder('category')
  .loadRelationIdAndMap('category.productsCount', 'category.products')
  .skip((page - 1) * limit)
  .take(limit)
  .getManyAndCount();

const data = rows.map((category) => ({
  ...category,
  productsCount: (category.productsCount as unknown as number[])?.length ?? 0,
}));
```

⚠ **Đã sửa so với bản gốc:** hàm "chuẩn sách giáo khoa" cho việc này tên là **loadRelationCountAndMap** (tự chạy **COUNT**, trả thẳng về số) — đây là hàm hầu hết tutorial/docs TypeORM nói tới. Nhưng ở bản TypeORM đang cài, hàm đó **không còn tồn tại** — bị bỏ hẵn, TS báo **Property 'loadRelationCountAndMap' does not exist**. Chỉ còn **loadRelationIdAndMap** — trả về **mảng id**, không phải số, phải tự **.length** ở tầng code như ví dụ trên.

Việc cần làm:

1. Tạo **Comment** entity + **CommentsModule** (dùng **nest g resource comments**)
2. Thiết lập quan hệ OneToMany/ManyToOne giữa Post và Comment — nhớ import **Comment** đầy đủ ở MỌI nơi dùng đến (mục 9.1)
3. **CommentsModule** đăng ký **TypeOrmModule.forFeature([Comment, Post])** — cần cả **Post** vì Service phải tự tra Post cha có tồn tại
4. Viết route **GET /posts/:id** trả về post kèm danh sách comment của nó (**relations: { comments: true }**)
5. Viết route lồng **POST /posts/:postId/comments** để tạo comment cho 1 post cụ thể
6. Thêm field **commentsCount** vào **GET /posts** (danh sách) bằng **loadRelationIdAndMap** + **.length** — xác nhận response không kéo theo nội dung comment nào

Authentication (JWT + Passport)

Mục tiêu: Hiểu toàn bộ luồng đăng ký → đăng nhập → cấp token → bảo vệ route bằng token.

10.1 Luồng tổng quan

1. POST /auth/register
 - hash password bằng bcrypt (KHÔNG BAO GIỜ lưu plain text password)
 - lưu user vào database
2. POST /auth/login
 - tìm user theo email
 - so sánh password gửi lên với hash đã lưu (bcrypt.compare)
 - nếu khớp → tạo JWT token (chứa payload: userId, email...) ký bằng secret key
 - trả token về client
3. Client lưu token, gửi kèm mọi request sau đó:
Header: Authorization: Bearer
4. Route được bảo vệ gắn @UseGuards(AuthGuard('jwt'))
 - PassportStrategy tự động verify chữ ký token bằng secret key
 - nếu hợp lệ → giải mã payload, gắn vào request.user
 - nếu không hợp lệ/hết hạn → tự động trả 401 Unauthorized

10.2 Vì sao dùng Passport?

`@nestjs/passport` là lớp tích hợp giữa NestJS và thư viện Passport.js (chuẩn công nghiệp cho authentication ở Node.js). Bạn định nghĩa một `JwtStrategy` (kế thừa `PassportStrategy`) khai báo cách lấy token (từ header), secret key để verify, và hàm `validate(payload)` chạy sau khi token đã được xác minh hợp lệ — trả về gì trong `validate()` thì thứ đó sẽ được gắn vào `request.user`.

10.3 Vì sao hash password bằng bcrypt

Bcrypt là hash **một chiều** (không giải mã ngược được) có "salt" ngẫu nhiên tích hợp — cùng 1 password hash 2 lần ra 2 chuỗi khác nhau, chống rainbow table. Khi login không "giải mã" hash, mà hash lại password vừa nhập rồi so sánh (`bcrypt.compare` làm việc này).

Liên hệ với AuthGuard giả lập ở Part 07: Guard ở Part 07 chỉ kiểm tra "có header Authorization hay không", không verify token. Part này thay nó bằng `AuthGuard('jwt')` của Passport (verify chữ ký JWT thật) — nên bỏ Guard giả đi khi đã có Guard thật, tránh nhầm lẫn.

Việc cần làm:

1. Tạo `User` entity, `AuthModule`
2. Cài `bcrypt`, `@nestjs/jwt`, `@nestjs/passport`, `passport-jwt`
3. Viết route `register` (hash password trước khi lưu) và `login` (verify + trả JWT)
4. Viết `JwtStrategy` + `JwtAuthGuard`, áp dụng lên các route cần đăng nhập (vd tạo/xoá post phải đăng nhập)

Testing với Jest

Mục tiêu: Viết unit test cho Service mà không cần khởi động cả app.

11.1 Unit test Service

Nest CLI tích hợp sẵn Jest. Với unit test, bạn dùng `Test.createTestingModule()` để tạo một "module giả lập" chỉ chứa đúng provider cần test — có thể **mock** Repository/dependency khác thay vì dùng database thật, giúp test chạy nhanh và độc lập.

11.2 e2e test

e2e (end-to-end) test khởi động toàn bộ app thật (dùng `supertest` để gửi HTTP request thật tới app trong bộ nhớ), kiểm tra toàn bộ luồng từ HTTP request tới response — gần với hành vi thực tế nhất nhưng chạy chậm hơn unit test.

Việc cần làm (tuỳ chọn, làm sau khi xong Part 1-10):

- Viết unit test cho `PostsService.findAll()` và `findOne()` với repository được mock
- Viết 1 e2e test đơn giản cho route `GET /posts`

Pagination

Mục tiêu: Không trả về toàn bộ bảng cùng lúc — chia trang, giới hạn số lượng bản ghi mỗi lần trả về client.

12.1 Vì sao cần pagination

Ở Part 8-9, `GET /posts` luôn trả **toàn bộ** bảng. Với 20 post thì không vấn đề gì, nhưng với 200,000 post — mỗi request sẽ kéo hết dữ liệu, tốn băng thông, tốn RAM, chậm cả server và client. Pagination giải quyết bằng cách chỉ trả về 1 "trang" (page) nhỏ mỗi lần, kèm thông tin để client biết còn bao nhiêu trang.

12.2 Query DTO cho pagination

```
import { Type } from 'class-transformer';
import { IsInt, IsOptional, Max, Min } from 'class-validator';

export class PaginationQueryDto {
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  page: number = 1;

  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)
  limit: number = 10;
}
```

Vì sao bắt buộc phải có `@Type(() => Number)` ở đây: query string trong URL (`?page=2&limit=10`) luôn được Express/Nest parse thành `string`, kể cả khi bạn khai `page: number` trong DTO. Khác với body JSON (nơi `class-transformer` tự suy ra kiểu số nếu JSON gửi số), query string cần `@Type(() => Number)` tường minh để `class-transformer` biết ép `"2"` thành `2` trước khi `class-validator` chạy `@IsInt()` — thiếu dòng này, `@IsInt()` luôn fail vì nhận vào string.

`@Max(100)` ở `limit` — chặn client tự ý xin `?limit=999999` (biến pagination thành vô nghĩa, lại quay về vấn đề tải hết bảng).

12.3 Dùng `skip / take` với TypeORM

```
async findAll(query: PaginationQueryDto) {
  const { page, limit } = query;
  const [data, total] = await this.postsRepository.findAndCount({
    skip: (page - 1) * limit,
    take: limit,
    order: { id: 'DESC' },
  });

  return {
    data,
    meta: { total, page, limit, totalPages: Math.ceil(total / limit) },
  };
}
```

`findAndCount()` chạy 2 việc trong 1 lần gọi: lấy đúng `limit` bản ghi bắt đầu từ vị trí `skip` (SQL `LIMIT / OFFSET`), và đếm tổng số bản ghi thoả điều kiện (không tính `LIMIT`) — cần cả 2 số này để tính `totalPages`. Nếu chỉ dùng `find({ skip, take })` thường, bạn sẽ không biết tổng cộng có bao nhiêu trang.

Route Controller — nhớ dùng `@Query()` , không phải `@Param()` :

```
@Get()
findAll(@Query() query: PaginationQueryDto) {
    return this.postsService.findAll(query);
}
```

`ValidationPipe` global (Part 5) tự áp dụng luôn cho `@Query()` , không cần cấu hình thêm — nhưng phải đảm bảo `transform: true` đã bật (mặc định của `ValidationPipe` khi bạn dùng `class-transformer`) để `@Type()` ở mục 12.2 thực sự chạy.

12.4 Tương tác với TransformInterceptor (Part 07)

`TransformInterceptor` đang bọc mọi response thành `{ success: true, data }` . Khi Service pagination tự trả về `{ data, meta }` , Interceptor sẽ bọc thêm 1 lớp thành `{ success: true, data: { data, meta } }` — lồng 2 lần chữ `data` , khó đọc. Không có đáp án "đúng tuyệt đối" — bạn tự chọn trade-off giữa "1 format nhất quán toàn app" (Part 07) và "pagination cần thêm field":

- **Cách 1:** Sửa Interceptor để nếu payload đã có sẵn field `meta` thì spread thẳng ra `{ success: true, data: result.data, meta: result.meta }` thay vì bọc thêm.
- **Cách 2:** Bỏ qua vấn đề, chấp nhận cấu trúc lồng — client đọc `response.data.data` và `response.data.meta` .

Việc cần làm:

1. Tạo `PaginationQueryDto` với `page` , `limit` có default + validate
2. Sửa `PostsService.findAll()` dùng `findAndCount()` với `skip / take`
3. Trả kèm `meta: { total, page, limit, totalPages }`
4. Tự quyết định và xử lý vấn đề lồng response với `TransformInterceptor` (mục 12.4)
5. Test `GET /posts?page=2&limit=5` , xác nhận đúng số lượng và đúng trang

PART 13 — MỚI

Transactions

Mục tiêu: Đảm bảo nhiều thao tác ghi database "thành công cùng nhau hoặc thất bại cùng nhau" (atomicity).

13.1 Vấn đề transaction giải quyết

Hình dung: tạo 1 `Post` mới, đồng thời cũng cần tạo sẵn 1 `Comment` chào mừng đầu tiên gắn với post đó. Nếu `save(post)` thành công nhưng `save(comment)` thất bại (lỗi mạng, lỗi validate, crash giữa chừng...) — bạn còn lại 1 `Post` "mồ côi" không có comment đi kèm như thiết kế, dữ liệu ở trạng thái nửa vời. Transaction đảm bảo: hoặc cả 2 lệnh cùng thành công (commit), hoặc cả 2 cùng bị hoàn tác (rollback) như chưa từng xảy ra.

13.2 Cách đơn giản — `DataSource.transaction()`

```
import { DataSource } from 'typeorm';

@Injectable()
export class PostsService {
  constructor(private readonly dataSource: DataSource) {}

  async createWithFirstComment(dto: CreatePostDto, firstComment: string) {
    return this.dataSource.transaction(async (manager) => {
      const post = await manager.save(PostEntity, dto);
      await manager.save(Comment, { content: firstComment, post });
      // throw ở BẤT KỲ đâu trong callback này → toàn bộ tự rollback
      return post;
    });
  }
}
```

`DataSource` (inject trực tiếp qua constructor, không cần `@InjectRepository`) đại diện cho kết nối database gốc. `.transaction(callback)` tự động: mở transaction (SQL `BEGIN`) → chạy callback → nếu callback chạy xong không lỗi thì `COMMIT` (ghi thật vào DB) → nếu callback `throw` bất kỳ đâu thì tự `ROLLBACK` (hoàn tác mọi thay đổi, như chưa từng chạy).

Lưu ý bắt buộc: mọi lệnh ghi database trong callback phải gọi qua `manager` được truyền vào (KHÔNG dùng `this.postsRepository.save()` như bình thường) — vì `manager` chính là "phiên làm việc" đang nằm trong transaction đó. Nếu lỡ tay dùng lại Repository thường bên ngoài, lệnh đó sẽ chạy trong 1 transaction/connection KHÁC, không được rollback cùng nhóm.

13.3 Cách kiểm soát tay — `QueryRunner`

```
async createWithFirstComment(dto: CreatePostDto, firstComment: string) {
  const queryRunner = this.dataSource.createQueryRunner();
  await queryRunner.connect();
  await queryRunner.startTransaction();

  try {
    const post = await queryRunner.manager.save(PostEntity, dto);
    await queryRunner.manager.save(Comment, { content: firstComment, post });
    await queryRunner.commitTransaction();
    return post;
  } catch (err) {
    await queryRunner.rollbackTransaction();
    throw err;
  } finally {
    // LUÔN release, kể cả khi lỗi — tránh rò rỉ connection
    await queryRunner.release();
  }
}
```

Cách này viết dài hơn `DataSource.transaction()` nhưng cho kiểm soát chi tiết hơn — cần khi bạn phải truyền cùng 1 `queryRunner` xuyên qua nhiều Service khác nhau (ví dụ 1 luồng nghiệp vụ gọi cả

`PostsService` và `CommentsService`, cả 2 phải nằm trong đúng 1 transaction).
`DataSource.transaction()` ở mục 13.2 không làm được việc này vì `manager` chỉ tồn tại trong phạm vi callback, không truyền tự nhiên qua lời gọi Service khác được.

13.4 So sánh nhanh

	<code>DataSource.transaction()</code>	<code>QueryRunner</code> thủ công
Độ dài code	Ngắn	Dài hơn, nhiều bước tay
Tự rollback khi throw	✅ Tự động	Phải tự viết try/catch/rollback
Dùng chung qua nhiều Service	Khó (manager giới hạn trong callback)	✅ Dễ — truyền <code>queryRunner</code> làm tham số
Phù hợp	Transaction đơn giản, gọn trong 1 Service	Transaction phức tạp, nhiều Service tham gia

Việc cần làm:

- Viết method `createWithFirstComment()` trong `PostsService` dùng `DataSource.transaction()`
- Test case thành công: gọi API, xác nhận cả `Post` và `Comment` đều xuất hiện trong database
- Test case rollback: chủ động sửa tạm code để `throw new Error('test rollback')` ngay sau khi save comment — gọi lại API, xác nhận **cả Post cũng KHÔNG được lưu** (kiểm tra bằng DevDb/SQLite Viewer hoặc `GET /posts`) — đây là bằng chứng transaction hoạt động đúng
- Xoá dòng `throw` test sau khi xác nhận xong